

DulceByte **Fase 3**

Evidencias de Aplicación Web - Spring Boot

Proyecto final MOD 3.3, 3.4, 3.5

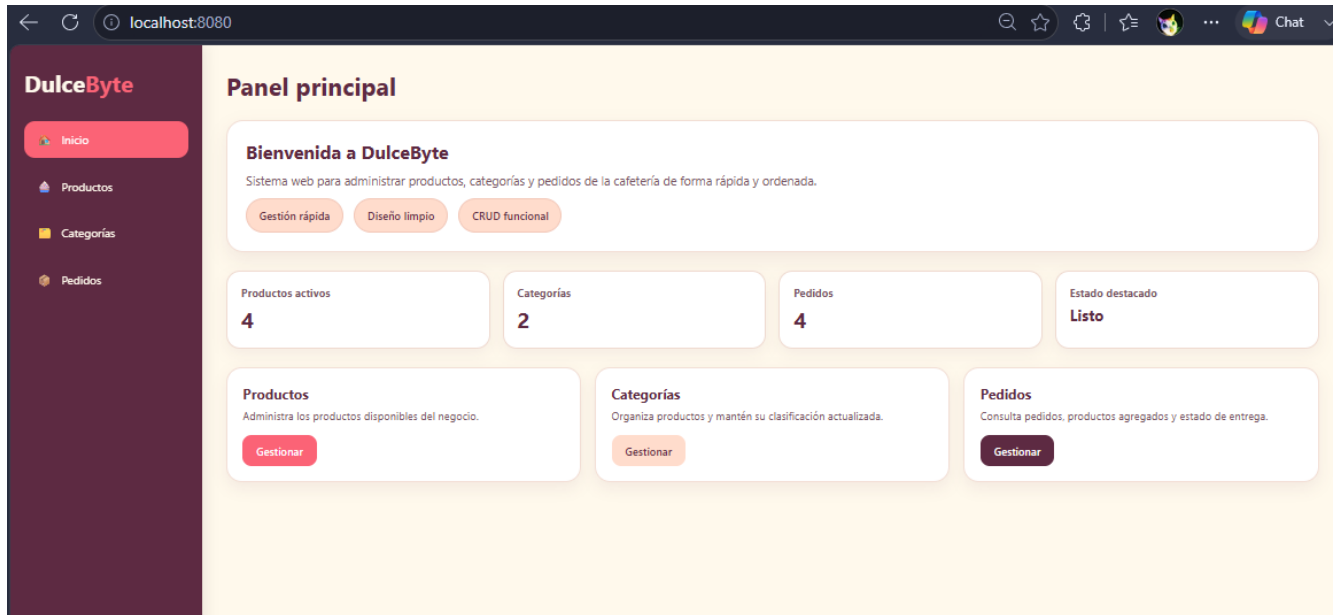
Aplicación web consumiendo la API REST de DulceByte

Arquitectura: Spring Boot -> API REST ASP.NET Core -> MySQL

Septiembre 2026

1. Aplicación web y navegación

Interfaz principal de DulceByte

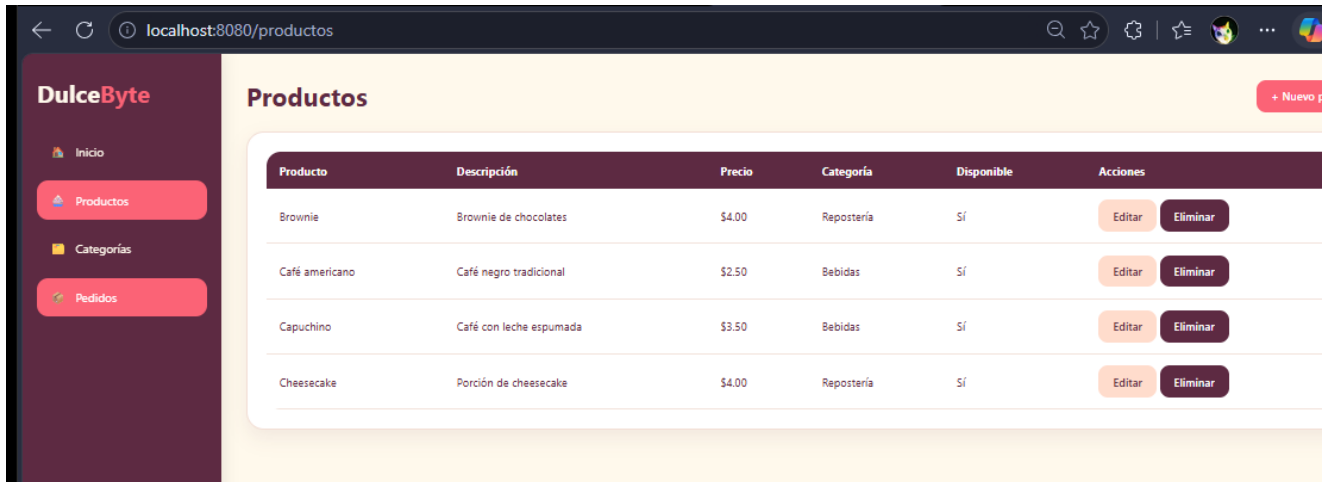


Panel principal con accesos a Productos, Categorías y Pedidos.

La aplicación web fue desarrollada con Spring Boot y Thymeleaf. La información mostrada proviene de la API REST de la Fase 2; Spring no accede directamente a MySQL.

2. Módulo CRUD - Productos

Consulta, registro, modificación y eliminación



Listado final de productos con acciones de edición y eliminación.

Productos DulceByte

Producto registrado correctamente

[+ Nuevo producto](#)

ID	Producto	Descripción	Precio	Categoría	Disponible	Acciones
6	BOOBA	Te	4.50	Bebidas	Si	Editar Eliminar
3	Brownie	Brownie de chocolates	4.00	Repostería	Si	Editar Eliminar
1	Café americano	Café negro tradicional	2.50	Bebidas	Si	Editar Eliminar
2	Capuchino	Café con leche espumada	3.50	Bebidas	Si	Editar Eliminar
4	Cheesecake	Porción de cheesecake	4.00	Repostería	Si	Editar Eliminar

Producto actualizado correctamente

[+ Nuevo producto](#)

ID	Producto	Descripción	Precio	Categoría	Disponible	Acciones
3	Brownie	Brownie de chocolates	4.00	Repostería	Si	Editar Eliminar
1	Café americano	Café negro tradicional	2.50	Bebidas	Si	Editar Eliminar
2	Capuchino	Café con leche espumada	3.50	Bebidas	Si	Editar Eliminar
4	Cheesecake	Porción de cheesecake	4.00	Repostería	Si	Editar Eliminar

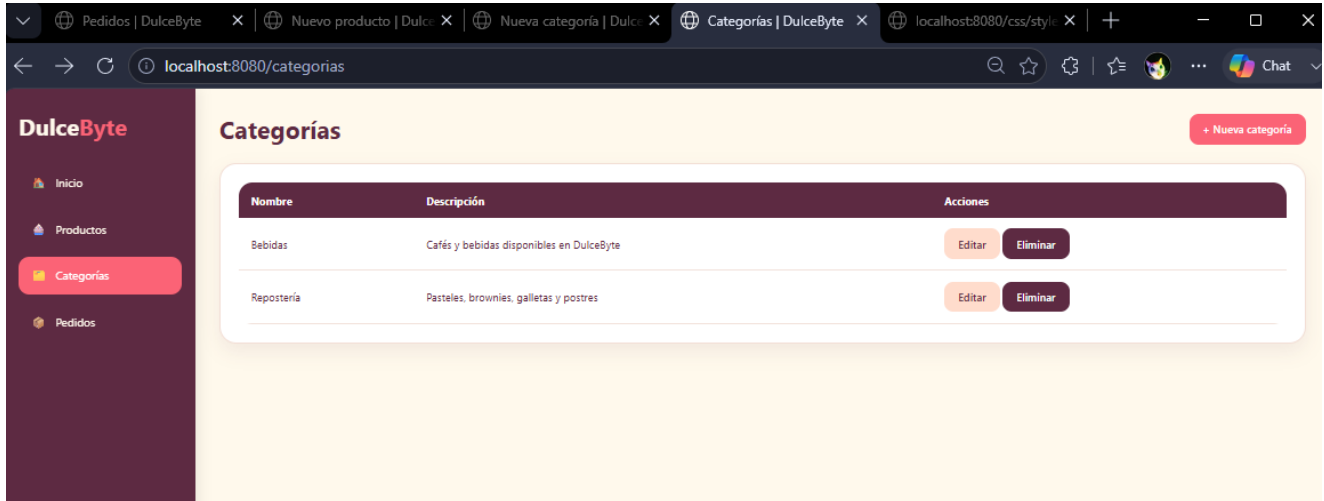
Actualización

Registro

Productos implementa las cuatro operaciones CRUD mediante peticiones HTTP a /api/productos. La eliminación fue verificada al retirar el producto temporal creado para la prueba.

3. Módulo CRUD - Categorías

Segundo módulo CRUD completamente funcional



Listado final de categorías con interfaz unificada.

Categorías DulceByte

Categoría registrada correctamente

[+ Nueva categoría](#)

ID	Nombre	Descripción	Acciones
1	Bebidas	Cafés y bebidas disponibles en DulceByte	Editar Eliminar
3	Juegos	Alquiler de juegos de mesa	Editar Eliminar
2	Repostería	Pasteles, brownies, galletas y postres	Editar Eliminar

[Ir a productos](#)

Registro

Categoría actualizada correctamente

[+ Nueva categoría](#)

ID	Nombre	Descripción	Acciones
1	Bebidas	Cafés y bebidas disponibles en DulceByte	Editar Eliminar
3	Juegos	Alquiler de juegos de computadora	Editar Eliminar
2	Repostería	Pasteles, brownies, galletas y postres	Editar Eliminar

[Ir a productos](#)

Actualización

Categorías DulceByte

Categoría eliminada correctamente

[+ Nueva categoría](#)

ID	Nombre	Descripción	Acciones
1	Bebidas	Cafés y bebidas disponibles en DulceByte	Editar Eliminar
2	Repostería	Pasteles, brownies, galletas y postres	Editar Eliminar

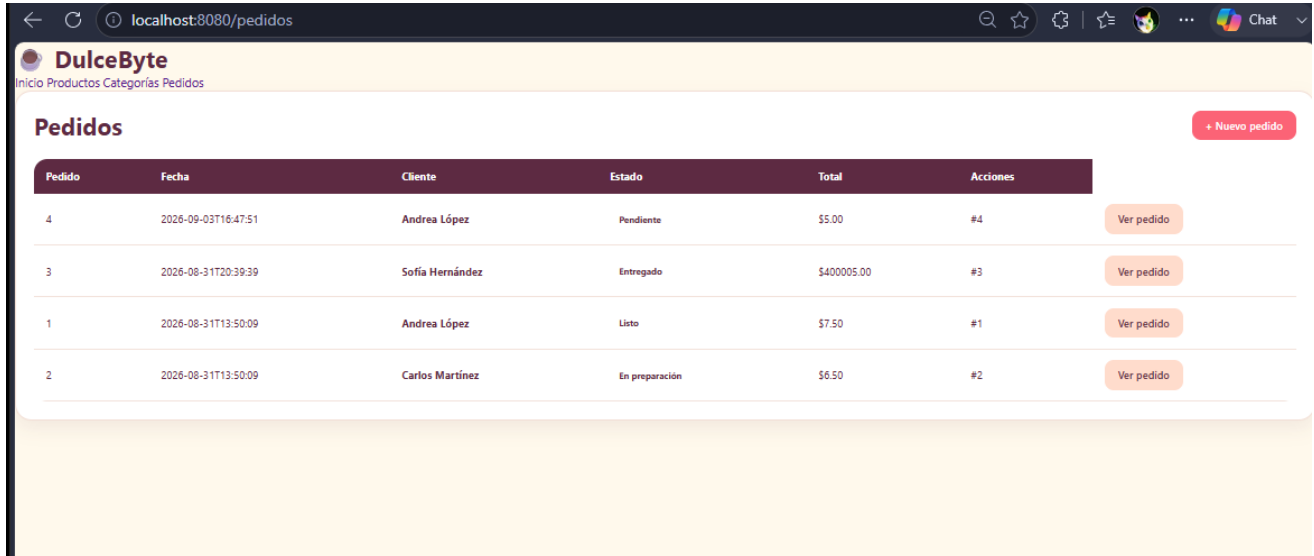
[Ir a productos](#)

Eliminación

Categorías completa el segundo CRUD: crear, consultar, modificar y eliminar registros mediante la API REST.

4. Operación principal del negocio - Pedidos

Flujo funcional de creación y gestión del pedido



Pedido	Fecha	Cliente	Estado	Total	Acciones
4	2026-09-03T16:47:51	Andrea López	Pendiente	\$5.00	#4 <button>Ver pedido</button>
3	2026-08-31T20:39:39	Sofía Hernández	Entregado	\$400005.00	#3 <button>Ver pedido</button>
1	2026-08-31T13:50:09	Andrea López	Listo	\$7.50	#1 <button>Ver pedido</button>
2	2026-08-31T13:50:09	Carlos Martínez	En preparación	\$6.50	#2 <button>Ver pedido</button>

Listado de pedidos con cliente, estado, total y acceso al detalle.

Pedido # 3

Cliente: Sofia Hernández

Estado: Listo

Total: \$5.00

Productos del pedido

Producto	Cantidad	Precio unitario	Subtotal
Café americano	2	2.50	5.00

Agregar producto

Producto Cantidad

Cambiar estado

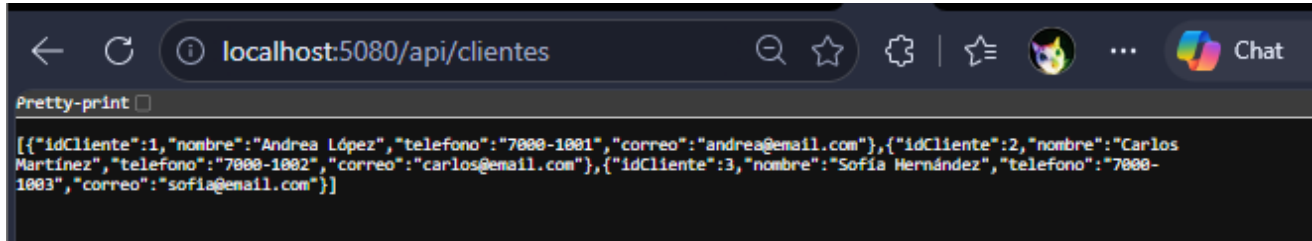
[← Volver a pedidos](#)

Detalle de un pedido: productos, total y cambio de estado.

El flujo principal permite crear pedidos, agregar productos, recalcular el total, consultar el detalle y actualizar el estado usando los endpoints de /api/pedidos.

5. Consumo de la API REST

Comunicación entre Spring Boot y ASP.NET Core

A screenshot of a web browser window. The address bar shows 'localhost:5080/api/clientes'. Below the address bar, there is a 'Pretty-print' button and a JSON response. The JSON response is an array of three objects, each representing a client with fields for id, name, phone, and email.

```
[[{"idCliente":1,"nombre":"Andrea L3pez","telefono":"7888-1881","correo":"andrea@email.com"}, {"idCliente":2,"nombre":"Carlos Mart3nez","telefono":"7888-1882","correo":"carlos@email.com"}, {"idCliente":3,"nombre":"Sof3a Hern3ndez","telefono":"7888-1883","correo":"sofia@email.com"}]]
```

Respuesta JSON de /api/clientes utilizada por la aplicaci3n web.

Los formularios recuperan informaci3n de la API y ejecutan operaciones HTTP. Durante las pruebas, la API se ejecuta en localhost:5080 y la aplicaci3n Spring en localhost:8080.

6. Verificación de requisitos de Fase 3

Requisito	Estado
Aplicación web con Spring / Spring Boot	Cumplido
Consumo de API REST desarrollada en Fase 2	Cumplido
Mostrar información proveniente de la API	Cumplido
CRUD completo #1: Productos	Cumplido
CRUD completo #2: Categorías	Cumplido
Operación principal del negocio: Pedidos	Cumplido
Registrar información	Cumplido
Modificar información	Cumplido
Eliminar información cuando corresponde	Cumplido
Mensajes de éxito / error	Cumplido
Navegación entre funcionalidades	Cumplido
Interfaz organizada y usable	Cumplido

Estructura de entrega: 03-Web/ProyectoSpring/ + 03-Web/EvidenciasWeb.pdf